

AgentIA — Revisión del sistema, verificación y coste

El flujo de trabajo agéntico, la orquestación, lo que se mide y lo que cuesta

Ingeniería AgentIA

2026-09-29

Qué es este documento. Una revisión del sistema tal como se ejecuta hoy: el flujo de trabajo agéntico de generación (el pipeline LangGraph y su bucle de verificación), la orquestación que lo rodea, el diseño, los resultados de verificación obtenidos esta semana y un informe de coste/uso recogido de los propios almacenes de la plataforma — el sistema de registro local en SQLite y su espejo MLflow.

Está escrito *después* de que la plataforma produjera sus primeras compilaciones verificadas, por lo que separa lo medido de lo afirmado. Donde las dos fuentes de coste discrepan, este documento lo dice en lugar de elegir una en silencio.

Contenido

1	Resumen ejecutivo	2
2	El flujo de trabajo agéntico	2
2.1	El pipeline como máquina de estados	2
2.2	Doble implementación: MODELO y DETERMINISTA	2
2.3	La compuerta de cumplimiento	2
2.4	El sandbox de verificación	3
2.5	Auto-reparación acotada	3
3	Orquestación	3
3.1	Dos puntos de entrada, un producto	3
3.2	Cola, ciclo de vida y streaming	3
3.3	Sustrato de medición y auto-mejora	3
4	Disponibilidad de la interfaz de usuario	4
5	Diseño	4
5.1	Inventario de servicios	4
6	Resultados de verificación	5
6.1	Escalado de cobertura: la palanca de fiabilidad	5
6.2	El verificador es intermitentemente inestable	5
7	Coste y uso	7
7.1	Sistema de registro (autoritativo)	7
7.2	Espejo MLflow (y dónde se ha desviado)	7
8	Hallazgos y próximos pasos	7

1 Resumen ejecutivo

Cuatro hechos dominan esta revisión.

1. **La plataforma ya verifica su propia salida.** Hasta esta semana el sistema reportaba “0 de 15 sesiones verificadas”. La causa fue un defecto de montaje Docker de un solo carácter (un reetiquetado SELinux privado concatenado a una bandera de solo-lectura, que producía una opción de montaje inválida y hacía que toda compilación fallara antes de ejecutar Maven). Está corregido, y las primeras compilaciones verificadas están registradas.
2. **El flujo de trabajo agéntico es real y está gobernado.** Una máquina de estados LangGraph ejecuta ocho nodos — validador, cinco etapas de generación, sandbox, reparación — con una compuerta de cumplimiento determinista en cada etapa y un bucle de reparación acotado. Cada etapa de generación tiene dos implementaciones, **MODELO** y **DETERMINISTA**, despachadas una vez por sesión.
3. **La selección guiada por verificación es la palanca de fiabilidad medida.** Con una sola muestra el generador compila el 78% de las veces; muestrear tres candidatos y quedarse con el primero que compila eleva la cobertura a nivel de tarea al 100% a \$0.20 por servicio fiable.
4. **El verificador es intermitentemente inestable, y el espejo de coste se ha desviado.** El mismo espacio de trabajo congelado recompila 4 de 6 veces bajo carga y 8 de 8 veces secuencialmente; la causa principal es la misma configuración de montaje. Y el espejo MLflow subestima el sistema de registro en $\approx \$1.6$ (20%), de modo que el almacén local — no el espejo — es la cifra a citar.

2 El flujo de trabajo agéntico

2.1 El pipeline como máquina de estados

La generación es una máquina de estados LangGraph de ocho nodos (`graph.py`). La topología es una columna lineal con exactamente un ciclo acotado, mostrada en la Figura 2:

```
validador → scaffolder → dominio → servicio → controlador → test → sandbox ⇌  
reparación
```

Cada nodo de generación emite uno o varios archivos en un espacio de trabajo compartido; el sandbox compila y prueba el proyecto completo de forma hermética; ante un fallo el nodo de reparación aplica un parche quirúrgico y el sandbox se vuelve a ejecutar, acotado por un número máximo de intentos.

2.2 Doble implementación: **MODELO** y **DETERMINISTA**

Cada etapa de generación tiene dos implementaciones (`stages/deterministic/` y `stages/model/`), seleccionadas una vez por sesión mediante `select_generation_mode()`. La decisión se registra en el estado y nunca se re-decide a mitad de sesión, de modo que una sesión no puede producir un híbrido de artefactos con forma de plantilla y con forma de modelo. La ruta determinista emite plantillas offline con cero llamadas al modelo y es idéntica byte a byte entre ejecuciones; la ruta de modelo conduce un LLM de uso general (actualmente `deepseek-flash`) con un conjunto de instrucciones versionado.

2.3 La compuerta de cumplimiento

Antes de persistir el candidato de cualquier etapa, pasa una compuerta de cumplimiento combinada (`stages/compliance.py`): se fusionan la familia de validación de análisis de tests y la familia de arquitectura de seguridad, y cada hallazgo lleva una atribución — **LOCAL** (esta etapa posee el artefacto infractor) o **ACUMULADO** (ninguna etapa lo posee). Los hallazgos locales bloqueantes rechazan el

candidato y consumen un intento de corrección; los hallazgos acumulados se registran pero nunca rechazan. Esta separación es lo que impide culpar a una sola etapa por una preocupación de todo el proyecto.

2.4 El sandbox de verificación

El sandbox (`workspace_verification.py`) ejecuta, en orden: eliminar los metadatos de control de versiones; inyectar un test de contrato de persistencia escrito por la plataforma (un `@SpringBootTest` con `ddl-auto=validate` contra el `schema.sql` generado); y luego `docker run -rm -network none ... mvn test -o` contra una caché Maven de solo lectura. El test de contrato es la señal de aceptación que el generador no puede autorar: el modelo nunca lo ve, y una contradicción esquema/entidad falla la compilación aunque todos los tests unitarios escritos por el modelo (que simulan el repositorio) pasen. Los conteos de tests se extraen del propio resumen de surefire en lugar de fabricarse.

2.5 Auto-reparación acotada

Ante un fallo de compilación, el nodo de reparación analiza la traza de Maven (`repair.py`, `repair_parser.py`) en una clase de fallo (compilación, aserción, tiempo de ejecución) y aplica un parche unificado quirúrgico, reentrando en el sandbox. El límite de intentos es 3 en la constitución y 5 en `config.py:73` — una inconsistencia viva señalada en la Sección 8.

3 Orquestación

3.1 Dos puntos de entrada, un producto

El sistema expone dos formas de iniciar una sesión, y esta revisión confirma que ambas ahora llegan al modelo y ambas verifican el espacio de trabajo:

- **Ruta de grafo** — `POST /sessions` encola una sesión cuyo worker ejecuta el grafo compilado de extremo a extremo, con las credenciales pasadas por la cabecera `X-LLM-API-Key`.
- **Ruta secuencial** — `POST /sessions/quick-start` con `autoRun`, y `/orchestrator/pipeline/run`, conducen `pipeline_runner`, que ejecuta las mismas etapas de generación, la misma verificación hermética, una auditoría de seguridad estática y las fases de DevOps/exportación.

La ruta secuencial antes generaba código y *nunca lo compilaba*; ahora ejecuta la misma veta de verificación que el grafo, de modo que las dos rutas no pueden divergir.

3.2 Cola, ciclo de vida y streaming

Una cola FIFO (`queue_service.py`) limita la concurrencia a dos ejecuciones de sandbox simultáneas. El ciclo de vida (`lifecycle_service.py`) recorre `REQUISITOS` → `ARQUITECTURA` → `MODELO_DATOS` → `CÓDIGO_TESTS` → `AUDITORÍA_SEGURIDAD` → `DEVOPS` → `EXPORTACIÓN` → `COMPLETADO`, con pausa, reanudación y cancelación en vuelo y streaming en vivo por Server-Sent Events a la pantalla de monitor.

3.3 Sustrato de medición y auto-mejora

Dos capas más acompañan a la generación. El instrumento de diagnóstico (`conformance_diagnostics.py`) es un puntuador determinista y atribuido por regla que persiste un registro duradero por sesión y produce un informe de corpus. El bucle de optimización de habilidades (features 014–015) está *construido* — recopilar, reflexionar, aplicar, una compuerta de mejora estricta, contribución por habilidad por leave-one-out y desalojo con suelo de evidencia — pero deliberadamente no se ejecuta, por las razones expuestas en la Sección 6.

4 Disponibilidad de la interfaz de usuario

La superficie de producto es una aplicación de página única React/Vite (la antigua app Streamlit se conserva como `frontend-legacy/`). Se organiza como un espacio de trabajo de diez pestañas con etiquetas en español, tabulado abajo; un stepper de ciclo de vida refleja la máquina de fases del backend, y los diagramas de arquitectura y entidad-relación se renderizan a través de un visor Mermaid.

Pestaña	Qué expone
0. Resumen	panel ejecutivo; inicio rápido 1-clic; modo Auto-Pilot vs paso a paso
1. Requisitos	historias de usuario BDD (Given/When/Then) y exportación <code>spec.md</code>
2. Arquitectura	arquitectura en 4 capas, catálogo de componentes, contratos REST (Mermaid)
3. Modelos & SQL	entidades JPA, registros DTO, <code>schema.sql</code> , diagrama ER
4. Blueprints	ingesta drag-and-drop de <code>spec.md</code> / JSON blueprint y validación
5. Monitor Live	logs en vivo por Server-Sent Events, progreso de fases, pausa/re-anudar/cancelar
6. Código & Fix	explorador de código, tests, diffs de reparación, intervención manual en sesiones bloqueadas
7. Calidad SAST	auditoría estática de secretos/CVE/arquitectura, compuerta de calidad, remediación 1-clic
8. DevOps & Demo	Dockerfile, compose, CI/CD, manifiestos k8s, despliegue local, smoke test, consola REST
9. Entrega Git	descarga ZIP y publicación Git atómica

Más allá de las pestañas: un panel de configuración LLM (proveedor: Gemini, Groq, OpenAI, DeepSeek o mock offline; clave API; modelo; verificación de conexión), autenticación en la pantalla de login, un conmutador de tema claro/oscuro y una lista de sesiones con estado. La interfaz refleja la verificación honesta de tres estados del backend (verificado / verificado con hallazgos / no evaluable), tras una auditoría de frontend que eliminó varios lugares donde la interfaz reportaba resultados que no habían ocurrido (un veredicto “UP” de smoke test fabricado, una consola REST que inventaba respuestas 200/201/204 y un formulario CRUD que inventaba un registro creado).

5 Diseño

5.1 Inventario de servicios

Servicio	Rol
<code>requirements_service</code>	descomposición de historias BDD y síntesis de spec
<code>architecture_service</code>	arquitectura en 4 capas y catálogo de componentes
<code>model_sql_service</code>	entidades JPA, DTOs, SQL de esquema/datos, diagrama ER
<code>test_analysis_service</code>	analizador estático de reglas constitucionales (familia A)
<code>security_service</code>	SAST, secretos, conformidad de arquitectura (familia B)
<code>conformance_diagnostics</code>	puntuador de conformidad determinista y atribuido por regla
<code>corpus_report</code>	informe de medición por lote/corpus

<code>workspace_verification</code>	eliminar VCS, inyectar test de contrato, ejecutar la compilación
<code>verification_selection</code>	best-of- k : generar, verificar, detenerse en el primer éxito
<code>platform_verification</code>	test de contrato de persistencia escrito por la plataforma
<code>skill_contribution</code>	contribución por habilidad por leave-one-out
<code>llm_factory</code>	detección de proveedor y construcción del cliente del modelo
<code>structured_output</code>	salida del modelo restringida por esquema
<code>injection_guard</code>	defensa contra prompt-injection del reflector
<code>queue_service /</code> <code>lifecycle_service</code>	conurrencia y orquestación de fases
<code>docker_service / devops /</code> <code>export / git</code>	despliegue en contenedor, CI, ZIP, publicación
<code>registro de coste + espejo</code> <code>mlflow</code>	coste por llamada y por sesión, espejo

6 Resultados de verificación

6.1 Escalado de cobertura: la palanca de fiabilidad

La selección guiada por verificación — generar k candidatos, verificar cada uno, quedarse con el primero que pasa — se mide aquí, no se toma prestada de otra clase de tarea. El brazo de control del experimento de esta semana son 6 tareas, cada una remuestreada 3–4 veces con las mismas instrucciones y modelo; la fórmula de cobertura es $\text{coverage}(k) = 1 - (1 - p)^k$ (*Large Language Monkeys*, arXiv:2407.21787).

Métrica	Valor
pass@1 (una muestra)	78.3%
cobertura $k=1$	83.3%
cobertura $k=2$	83.3%
cobertura $k=3$	100.0%
cobertura $k=4$	100.0%

A \$0.065 por servicio generado, tres candidatos cuestan **\$0.20 por servicio fiable**. Advertencias, declaradas y no suavizadas: $n=6$ tareas, un modelo, un proveedor; esto es una demostración de la palanca, no un benchmark, y la dirección (la cobertura es monótona en k) es robusta mientras que el 100% es “6 de 6 tareas”.

6.2 El verificador es intermitentemente inestable

El mismo espacio de trabajo congelado, recompilado bajo carga antes, pasó 4 de 6 veces; recompilado secuencialmente, dos espacios de trabajo pasaron 8 de 8 cada uno. El fallo, cuando ocurre, es una `PluginContainerException` de surefire sobre un import foráneo de `surefire-shared-utils` — después de que todos los tests hayan pasado.

La inestabilidad no es, por tanto, un defecto de código por espacio de trabajo. Es intermitente y correlacionada con la carga, y la hipótesis principal es la configuración de montaje: `DOCKER_MOUNT_SUFFIX=:Z` es un reetiquetado SELinux *privado*, correcto para un espacio de trabajo por sesión pero incorrecto para una *caché Maven compartida de solo lectura* montada por muchos contenedores concurrentes. Dos contenedores reetiquetando una fuente compartida con `:Z` es una causa conocida de exactamente esta clase de fallo intermitente de realm de clases. La corrección propuesta es aplicar `:Z`

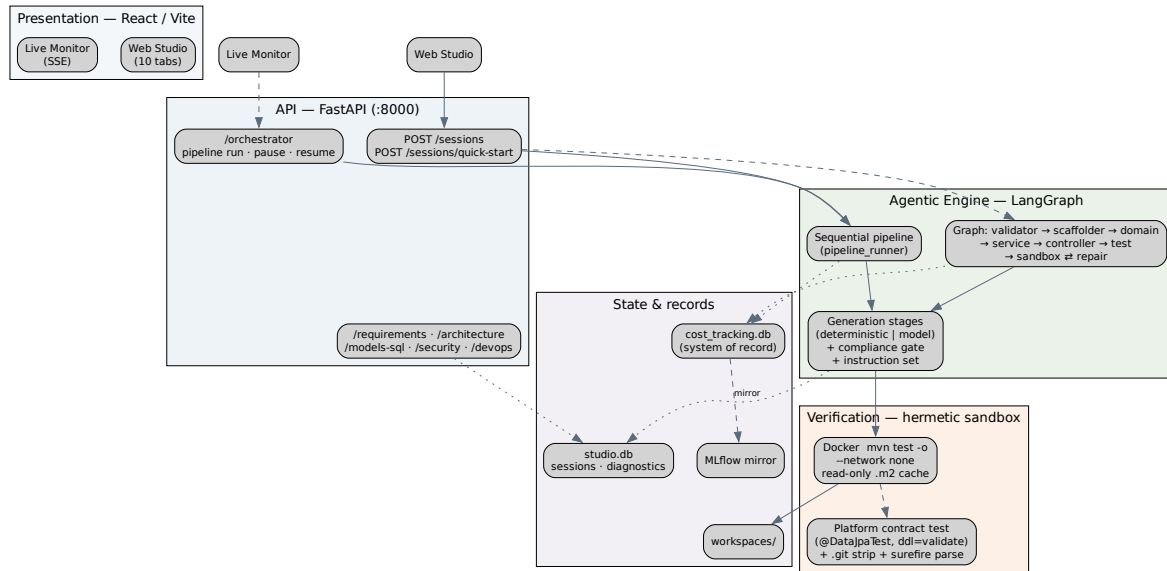


Figura 1: Arquitectura del sistema: el estudio React sobre la API FastAPI, el motor agéntico LangGraph, el sandbox de verificación hermético y los almacenes de estado/coste.

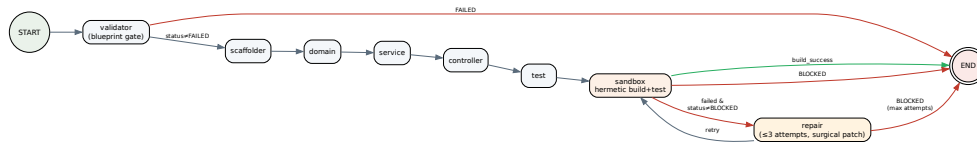


Figura 2: La máquina de estados de generación: una columna lineal con un ciclo acotado de reparación-sandbox.

solo al espacio de trabajo y dejar la caché sin reetiquetar (o :z), confirmada por una compuerta de repetibilidad.

Por qué esto condiciona todo lo posterior. Un verificador que se voltea $\approx 33\%$ de las ejecuciones no puede servir como oráculo de admisión: la cobertura prometería de más, una compuerta de admisión promovería ruido, y el teorema de Ratchet ([arXiv:2605.22148](#)) dice que un juez tan ruidoso no retira nada a ningún tamaño de muestra. Hacer el oráculo determinista es, por tanto, el prerequisite de toda medición posterior — incluido el bucle de optimización de habilidades, que permanece cerrado tras él.

7 Coste y uso

Los costes se registran por llamada y por sesión en el almacén SQLite local (`backend/cost_tracking.db`), que es el **sistema de registro**, y se espejan en MLflow (`mlflow.db`). Ambos se comparan aquí, no se fusionan.

7.1 Sistema de registro (autoritativo)

Alcance	Llamadas	Coste (USD)	Tokens salida
Experimento de polaridad (p4)	750	6.58	—
Línea base cinco congelados (corpus)	120	0.63	—
Diez difíciles	50	0.64	—
Sondas / sesiones sueltas	≈ 250	≈ 0.16	—
Total	1172	8.01	12.28M

Las 1172 llamadas fueron `deepseek-flash`. Total de tokens de entrada 6.45M, de salida 12.28M. El coste dominante es el experimento de polaridad de instrucciones, que es una medición terminada (un nulo), no un drenaje continuo.

7.2 Espejo MLflow (y dónde se ha desviado)

Experimento	Runs	Coste (USD)	Tok entr.	Tok sal.
Default (legado)	877	5.41	4.63M	8.20M
agentia (actual)	114	0.98	0.78M	1.52M
Total espejo	991	6.39	5.40M	9.72M

El espejo se ha desviado. MLflow totaliza **\$6.39** frente a los **\$8.01** del sistema de registro — una subestimación de $\approx \$1.62$ (20%), con ≈ 181 llamadas menos y totales de tokens que también discrepan. El espejo está dividido entre un experimento legado y uno actual, y solo se escriben métricas por llamada (faltan los agregados de sesión). La afirmación anterior de que “el espejo funciona y coincide” ya no es cierta para los datos actuales. Hasta que se repare el sink, el almacén local es la única cifra a citar, y un informe de coste generado solo desde MLflow subestimaría el gasto.

8 Hallazgos y próximos pasos

1. **Hacer determinista el verificador antes de gastar más.** Separar el sufijo de montaje (espacio de trabajo :Z, caché compartida sin reetiquetar) y añadir una compuerta de repetibilidad: un espacio de trabajo congelado debe recompilar N/N veces.

2. **Hacer que la medida pueda moverse.** El instrumento de conformidad lee 0 en compilaciones que fallan; exponer las clases de fallo del analizador de reparación como hallazgos acumulados para que un fallo de compilación sea atribuible.
3. **Usar la selección guiada por verificación como mecanismo de fiabilidad ya.** Está medida (\$0.20/servicio a $k=3$), no necesita entrenamiento y es la palanca que la literatura sitúa en primer lugar.
4. **Reparar el espejo de coste.** El espejado a nivel de sesión no se está poblando; restaurarlo o la UI de MLflow subestima el gasto.
5. **Resolver la inconsistencia del límite de reparación** (constitución 3 vs `config.py` 5).
6. **Dejar cerrado el bucle de optimización de habilidades** hasta que los puntos 1 y 2 pasen sus compuertas; el experimento de polaridad de instrucciones que era su entrada más prometedora es un nulo (prohibición vs control: 1 victoria, 3 derrotas, 2 empates).